

ユーザー登録画面の実装

1. entity/UserAccount.java

ここまでに記述してきたコードで既に完成されています。 `@PrePersist` による `accountId` の自動生成ロジックがあるため、登録時に ID を手動で発行する必要がないのが大きなメリットです。

2. repository/UserAccountRepository.java

ログイン用の ID（メールアドレス）が `userId` なので、メソッド名もそれに合わせます。

```
public interface UserAccountRepository extends JpaRepository<UserAccount, String> {  
    // メールアドレス (userId) での重複チェック  
    boolean existsByUserId(String userId);  
  
    // ログイン時に使用  
    Optional<UserAccount> findByUserId(String userId);  
}
```

3. service/UserAccountService.java

以下のユーザー新規登録機能を実装します。

```
@Service
public class UserAccountService {
    @Autowired
    private UserAccountRepository userAccountRepository;
    @Autowired
    private PasswordEncoder passwordEncoder;

    public void registerNewUser(UserAccount user) {
        // 1. 重複チェック
        if (userAccountRepository.existsByUserId(user.getUserId())) {
            throw new RuntimeException("このメールアドレスは既に登録されています。");
        }
        // 2. パスワードのハッシュ化
        user.setPassword(passwordEncoder.encode(user.getPassword()));
        // 3. デフォルト値のセット（必要に応じて）
        user.setIsAuth(0); // 一般ユーザーとして登録
        user.setQuitDemand(0); // 退会フラグはオフ
        // 4. 保存（accountIdとcreatedAtは@PrePersistで自動生成される）
        userAccountRepository.save(user);
    }
}
```

注意点 (1)

UserAccountService.java の先頭部では、次のimport文が記述されていることを確認してください。これらが抜けていると、SpringBootの起動時にエラーが発生する原因となってしまいます。

```
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.security.crypto.password.PasswordEncoder;  
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
```

4. controller/UserAccountController.java

エンドポイントを作成します。フロントエンドから「ユーザー名」「メールアドレス (userId)」「パスワード」が送られてくることを想定します。

```
@RestController
@RequestMapping("/api/auth")
public class UserAccountController {
    @Autowired
    private UserAccountService userAccountService;

    @PostMapping("/register")
    public ResponseEntity<String> register(@RequestBody UserAccount user) {
        try {
            userAccountService.registerNewUser(user);
            return ResponseEntity.ok("登録が完了しました。");
        } catch (Exception e) {
            // 重複エラーなどを400(Bad Request)で返す
            return ResponseEntity.badRequest().body(e.getMessage());
        }
    }
}
```

5. config/SecurityConfig.java

GitHub Codespaces環境でのCORS設定や、認証の除外設定を確認します。

- **許可設定:** `.requestMatchers("/api/auth/register").permitAll()` を追加し、ログイン前でも登録できるようにします。
- **Bean定義:** `BCryptPasswordEncoder` が Bean として登録されているか確認してください。

```
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder; // 追加
import org.springframework.security.crypto.password.PasswordEncoder;      // 追加

@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

解説 ～SecurityConfig.javaへの定義追加について～

SecurityConfigで `@Bean` として `PasswordEncoder` を定義（登録）することで、Spring Bootという「工場」の中にその部品が置かれます。そうするとService側で `@Autowired` と書くだけで、工場からその部品が自動的に運ばれてきて使えるようになる、という仕組みです。

注意点 (2)

`UserAccount` エンティティには `about` や `lastLoginAt` など、登録時には入力しない（あるいは Null で良い）フィールドがいくつかあります。

- **フロントエンドとの整合性:** フロント側から送る JSON に `userName` , `userId` , `password` だけが含まれていれば、Spring Boot が自動的に `UserAccount` オブジェクトへマッピングしてくれます。
- **パスワード確認:** 画面構成案にあった「パスワード（確認用）」は、サーバーに送る前に **フロントエンド側で一致チェック** を行い、一致した場合に `password` としてバックエンドに送ります。

フロントエンド側の実装

1. RegisterForm.vue の作成

まずは、新規ユーザー登録を行うためのコンポーネントを作成します。`src/components/` 配下に `RegisterForm.vue` という名前でファイルを作成してください。

テンプレート（HTML構造）のポイント

画面構成案に基づき、以下の入力欄を用意します。

- ユーザー名（表示名）
- ユーザーID（メールアドレス）
- パスワード
- パスワード（確認用）


```
<template>
  <div class="register-form">
    <h2>ユーザー登録</h2>
    <form @submit.prevent="handleRegister">
      <div>
        <label>ユーザー名:</label>
        <input v-model="userName" type="text" required placeholder="例：田中 太郎" />
      </div>
      <div>
        <label>ユーザーID (メールアドレス):</label>
        <input v-model="userId" type="email" required placeholder="example@mail.com" />
      </div>
      <div>
        <label>パスワード:</label>
        <input v-model="password" type="password" required />
      </div>
      <div>
        <label>パスワード (確認用):</label>
        <input v-model="passwordConfirm" type="password" required />
      </div>

      <div v-if="errorMessage" class="error">{{ errorMessage }}</div>

      <button type="submit">これで登録する</button>
      <button type="button" @click="resetForm">リセット</button>
    </form>
    <p @click="$emit('switch-to-login')" style="cursor:pointer; color:blue;">
      既にアカウントをお持ちの方はこちら（ログイン画面へ）
    </p>
  </div>
</template>
```

2. ロジックの実装（<script setup>）

次に、入力されたデータの管理と、バックエンドへの送信ロジックを書きます。

```
<script setup>
import { ref } from 'vue';

const userName = ref('');
const userId = ref('');
const password = ref('');
const passwordConfirm = ref('');
const errorMessage = ref('');

const emit = defineEmits(['register-success', 'switch-to-login']);

const handleRegister = async () => {
  // 1. パスワード一致チェック
  if (password.value !== passwordConfirm.value) {
    errorMessage.value = "パスワードが一致しません。";
    return;
  }
}
```

```
try {
  const response = await fetch('https://あなたのCodespacesのURL/api/users/register', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      userName: userName.value,
      userId: userId.value,
      password: password.value
    })
  });

  if (response.ok) {
    alert("登録が完了しました！ログインしてください。");
    emit('switch-to-login'); // 成功したらログイン画面に切り替える
  } else {
    const errorText = await response.text();
    errorMessage.value = errorText || "登録に失敗しました。";
  }
} catch (error) {
  errorMessage.value = "サーバーとの通信に失敗しました。";
}

};

const resetForm = () => {
  userName.value = '';
  userId.value = '';
  password.value = '';
  passwordConfirm.value = '';
  errorMessage.value = '';
};
</script>
```

3. App.vue での表示切り替え

「ログイン画面」を表示する App.vue を編集し、「登録画面」と切り替えられるようにします。

App.vue のテンプレート構成例

現在の App.vue の構造に合わせて、以下のような v-if / v-else-if / v-else のブロックを構成してください。

```
<template>
  <div id="app">
    <header v-if="user" class="app-header">
      <div class="logo">勤怠システム</div>
      <div class="menu-container">
        <button @click="isMenuOpen = !isMenuOpen" class="hamburger">
          <span></span><span></span><span></span>
        </button>
      </div>
    </header>
  </div>
</template>
```

```
<div v-if="isMenuOpen" class="dropdown-menu">
  <p class="user-info">{{ user.userName }} さん</p>
  <hr>
  <button @click="logout" class="logout-btn">ログアウト</button>
</div>
</div>
</header>
<main>
  <AttendanceBoard
    v-if="user"
    :accountId="user.accountId"
    :userName="user.userName"
  />

  <RegisterForm
    v-else-if="isRegisterMode"
    @register-success="isRegisterMode = false"
    @switch-to-login="isRegisterMode = false"
  />

  <div v-else>
    <LoginForm @login-success="handleLoginSuccess" />
  </div>
</main>
</div>
```

```
    <div class="switch-mode-link">
      <p>アカウントをお持ちでないですか？</p>
      <button @click="isRegisterMode = true" class="text-button">新規ユーザー登録</button>
    </div>
  </div>
</main>
</div>
</template>
```

```
<!-- <script setup>については省略 -->
```

```
<style scoped>
.switch-mode-link {
  margin-top: 20px;
  text-align: center;
}
.text-button {
  background: none;
  border: none;
  color: #007bff;
  text-decoration: underline;
  cursor: pointer;
  padding: 5px;
}
.app-header { display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 10px 20px;
  background: #333; color: white;
}
```

```
.hamburger { background: none;
border: none;
cursor: pointer;
display: flex;
flex-direction: column;
gap: 5px;
}
.hamburger span { display: block; width: 25px; height: 3px; background: white; }
.menu-container { position: relative; }
.dropdown-menu {
position: absolute;
right: 0; top: 40px;
background: white;
color: #333;
border: 1px solid #ddd; border-radius: 5px;
padding: 10px;
min-width: 150px;
z-index: 100;
box-shadow: 0 4px 6px rgba(0,0,0,0.1);
}
.logout-btn { width: 100%;
padding: 8px;
background: #f44336; color: white;
border: none; border-radius: 3px;
cursor: pointer;
}
.user-info { font-size: 0.9rem; margin: 5px 0; }
</style>
```

修正のポイント

優先順位の整理

- まず `user` が存在するか（ログイン済みか）を最優先でチェックしています。ログインしていればヘッダーと `AttendanceBoard` が表示されます。

`v-else-if="isRegisterMode"`

- ログインしていない場合に、次に「登録モードかどうか」を判定します。ここで新規作成する `RegisterForm` を表示します。

ログイン画面＋切替ボタン

- 最後の `v-else` （未ログインかつ登録モードでもない）で `LoginForm` を表示します。
- ログイン画面の下に `isRegisterMode = true` に変えるボタンを配置することで、ユーザーが登録画面へ遷移できるようにしています。

イベントハンドリング

- `RegisterForm` から「登録成功」や「戻る」のイベントが飛んできたときに `isRegisterMode = false` にすることで、スムーズにログイン画面へ戻るようにしています。

事前準備の再確認

このテンプレートを動かすために、`<script setup>` 内で以下の2点を確認してください。

- `import RegisterForm from './components/RegisterForm.vue';` が記述されていること。
- `const isRegisterMode = ref(false);` が定義されていること。

これで画面の切り替えロジックは完璧です。まずはこの構成で、ログイン画面の下にボタンが表示され、登録画面へ切り替わるか試してみてください。